

```
import pandas as pd
import numpy as np

from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import BernoulliRBM
from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
```

```
# Upload file in Colab first
df = pd.read_excel('/content/Online Retail.xlsx')

df.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom

```
# Remove missing values
df = df.dropna()

# Remove negative or zero quantity
df = df[df['Quantity'] > 0]

# Remove negative price
df = df[df['UnitPrice'] > 0]

# Convert InvoiceDate
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

df.shape
```

(397884, 8)

```
# Convert CustomerID to string
df['CustomerID'] = df['CustomerID'].astype(str)

# Create product ID
df['StockCode'] = df['StockCode'].astype(str)
```

```
basket = df.pivot_table(index='CustomerID',
                        columns='StockCode',
                        values='Quantity',
                        aggfunc='sum',
                        fill_value=0)

basket.head()
```

StockCode	10002	10080	10120	10123C	10124A	10124G	10125	10133	10135	11001	...	90214V	90214W	90214Y	90214Z	...
CustomerID																
12346.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	...
12347.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	...
12348.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	...
12349.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	...
12350.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	...

5 rows × 3665 columns

```
basket_binary = basket.copy()

basket_binary[basket_binary > 0] = 1

basket_binary.head()
```

StockCode	10002	10080	10120	10123C	10124A	10124G	10125	10133	10135	11001	...	90214V	90214W	90214Y	90214Z	CHAR
CustomerID																
12346.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	
12347.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	
12348.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	
12349.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	
12350.0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	

5 rows × 3665 columns

```
scaler = MinMaxScaler()
basket_scaled = scaler.fit_transform(basket_binary)
```

```
X_train, X_test = train_test_split(basket_scaled, test_size=0.2, random_state=42)
```

```
rbm = BernoulliRBM(n_components=50, # hidden neurons
                  learning_rate=0.01,
                  n_iter=20,
                  random_state=42,
                  verbose=True)
```

```
rbm.fit(X_train)
```

```
[BernoulliRBM] Iteration 1, pseudo-likelihood = -1082.13, time = 2.30s
[BernoulliRBM] Iteration 2, pseudo-likelihood = -590.40, time = 1.24s
[BernoulliRBM] Iteration 3, pseudo-likelihood = -305.85, time = 1.36s
[BernoulliRBM] Iteration 4, pseudo-likelihood = -296.01, time = 1.20s
[BernoulliRBM] Iteration 5, pseudo-likelihood = -284.03, time = 1.20s
[BernoulliRBM] Iteration 6, pseudo-likelihood = -283.83, time = 1.26s
[BernoulliRBM] Iteration 7, pseudo-likelihood = -285.17, time = 1.37s
[BernoulliRBM] Iteration 8, pseudo-likelihood = -280.31, time = 1.28s
[BernoulliRBM] Iteration 9, pseudo-likelihood = -274.02, time = 2.54s
[BernoulliRBM] Iteration 10, pseudo-likelihood = -263.43, time = 1.35s
[BernoulliRBM] Iteration 11, pseudo-likelihood = -255.39, time = 1.21s
[BernoulliRBM] Iteration 12, pseudo-likelihood = -251.85, time = 1.19s
[BernoulliRBM] Iteration 13, pseudo-likelihood = -247.60, time = 1.21s
[BernoulliRBM] Iteration 14, pseudo-likelihood = -245.59, time = 1.15s
[BernoulliRBM] Iteration 15, pseudo-likelihood = -247.09, time = 1.14s
[BernoulliRBM] Iteration 16, pseudo-likelihood = -248.41, time = 1.16s
[BernoulliRBM] Iteration 17, pseudo-likelihood = -244.20, time = 1.22s
[BernoulliRBM] Iteration 18, pseudo-likelihood = -245.92, time = 2.75s
[BernoulliRBM] Iteration 19, pseudo-likelihood = -243.99, time = 1.38s
[BernoulliRBM] Iteration 20, pseudo-likelihood = -245.60, time = 1.23s
```

▼

BernoulliRBM

[i](#) [?](#)

```
BernoulliRBM(learning_rate=0.01, n_components=50, n_iter=20, random_state=42,
             verbose=True)
```

```
train_transformed = rbm.transform(X_train)
test_transformed = rbm.transform(X_test)

print("Transformed Shape:", train_transformed.shape)
```

Transformed Shape: (3470, 50)

```
plt.plot(rbm.components_.mean(axis=1))
plt.title("RBM Hidden Feature Activation")
plt.xlabel("Hidden Units")
plt.ylabel("Activation")
plt.show()
```

